E-Commerce Analytics Project Analysis Report (Python + MySQL)

Project Overview

This report analyzes the submitted Python notebook and MySQL query script for the e-commerce analytics project. The project uses Python (pandas, matplotlib, seaborn, mysql-connector-python) with a MySQL database named ecommerce. CSV datasets are imported into relational tables and analytical SQL queries are executed through Python for reporting and visualization.

Technical Stack

Python libraries used: pandas, mysql.connector, matplotlib, seaborn. Database: MySQL Workbench / MySQL Server. Main tables referenced: customers, orders, payments, products, order_items, sellers, geolocation.

Architecture Summary

The notebook first connects Python to MySQL, loads CSV files into database tables, executes SQL analytics queries, converts results into pandas DataFrames, and visualizes selected outputs using bar charts and line charts.

1. Top 10 Customer Cities by Number of Customers

Counts customers grouped by city to identify strongest customer concentration markets.

2. Orders Placed in a Specific Year

Counts orders placed in 2018 (Python also checks 2017 in notebook) using YEAR(order_purchase_timestamp).

3. Sales by Product Category

Joins products, order_items, and payments to determine highest revenue categories.

4. Installment Payment Percentage

Measures percentage of orders where payment_installments > 1 to understand payment behavior.

5. Customers by State

Aggregates customer distribution state-wise; notebook visualizes top states.

6. Monthly Order Trend

Monthly aggregation for 2018 order volume to identify seasonal trends.

MAYURESH

7. Average Products per Order by City

Uses CTE count_per_order to compute average basket size city-wise.

8. Revenue Contribution by Category

Calculates each category's percentage share of total revenue.

9. Product Price vs Purchase Frequency / Seller Revenue

Notebook content partly overlaps; SQL file labels correlation but actual logic ranks seller revenue.

10. Seller Revenue Ranking

Uses DENSE_RANK window function to rank sellers by total payment revenue.

11. Moving Average of Customer Order Value

Uses window frame of 3 rows (2 preceding + current) for rolling customer spend trend.

12. Cumulative Monthly Sales Across Years

Window SUM tracks revenue growth progression month-by-month.

13. Year-over-Year Growth Logic

Notebook additionally uses LAG to compare annual payment growth.

14. Customer Retention Within 6 Months

Uses first purchase date and checks repeat orders inside six months.

15. Top 3 Customers by Year

Ranks highest spenders annually using partitioned DENSE_RANK.

MAYURESH

Important Technical Findings

• Several notebook cells repeat database connection code; this can be modularized into reusable functions.

• Database credentials are hardcoded in notebook; production projects should use environment variables.

• SQL Query #9 title and implementation do not match: title mentions price correlation but query computes seller revenue.

• Query numbering skips #12 in SQL file; documentation should maintain sequential consistency.

• Retention formula denominator appears inverted and may need verification depending on intended metric definition.

• Some chart logic uses sorting after fetch, which is good practice for readability.

Professional Improvement Recommendations

• Create a config file for database connection.

• Wrap query execution in Python functions for reusability.

• Add exception handling for failed connections and SQL execution.

• Document each SQL query with business objective, expected output, and visualization type.

• Separate ETL, SQL analytics, and plotting into dedicated notebook sections.

MAYURESH